Product Aesthetic Design: A Machine Learning Augmentation

Detailed Literature Review: Burnap, Hauser, and Timoshenko (2023)

Runming Huang & Leran Li

February 23, 2026

Introduction & Motivation

The Theoretical Gap: Gestalt vs. Attributes

- **Traditional Marketing Approach:** Typically relies on *Conjoint Analysis*, where products are broken down into discrete attributes (e.g., horsepower, price, fuel efficiency).
- **The Aesthetic Challenge:** Beauty is *Gestalt*—it is an integrated whole. You cannot simply sum up "headlight shape + grille size" to get the "beauty score."
- **Data Paradox:**
  - Managers know design drives sales, but they lack a "ruler" to measure it.
  - Existing computer vision models (like ImageNet-trained CNNs) focus on *classification* (is this a car?), not *valuation* (is this a *beautiful* car?).

The Objective

Bridge the gap between "unstructured visual data" and "structured marketing decisions" using a unified machine learning framework.

The Practical Crisis in Industry Design

- **The "Aesthetic Dilution" Process:**
  1. Designers create bold, visionary sketches.
  2. Engineering constraints (aerodynamics, safety) and financial constraints (manufacturing costs) force modifications.
  3. Without a quantitative measure of "lost beauty," the design is often compromised to the point of losing its market appeal.
- **The Cost of Feedback:** Because human evaluation is so expensive ($\$100k$ per clinic), designers only get feedback *at the very end* of the process, when it's often too late to make major changes.

Research Question

Can we build an "Aesthetic Co-pilot" that gives designers real-time feedback and helps them navigate the tradeoff between creativity and engineering constraints?

Methodology: The AI Engine

Overcoming the "Thin Data" Barrier

The biggest technical hurdle in this paper is the **data imbalance**:

Unlabeled Data ($N \approx 180,000$)

- "Easy" to get from web scraping.
- Teaches the model *Feature Representation*: what do cars generally look like?

Labeled Data ($N = 203$)

- "Hard" and expensive to get.
- Teaches the model *Aesthetic Valuation*: which features do consumers prefer?

The Solution: Semi-supervised Learning. The model "pre-trains" its visual cortex on the large dataset, then "fine-tunes" its aesthetic sense on the small dataset.

The Five Pillars of the Model Architecture

The system is a **Multi-component Deep Neural Network**:

1. **Encoder (E):** Compresses the pixels into a latent vector z (the "Aesthetic Embedding").
2. **Generator (G):** Takes z and creates a high-resolution image. It is the "creative hand" of the AI.
3. **Discriminator (D):** The "critic" that tells G if the car looks realistic or "fake." (GAN component).
4. **Predictor (P):** The "consumer simulator" that estimates the aesthetic score from z.
5. **Attribute Classifier (C):** Predicts metadata (brand, year) from z. This allows the model to work even when some data (like the brand of a scraped image) is missing.

Mathematical Logic: The Joint Embedding Space

The paper doesn't just encode images; it encodes **Images (X) + Attributes (a)**:

- **Conditional Variational Autoencoder (VAE):** The latent space z is conditioned on a. This means the model knows that a "2020 Cadillac" should look different from a "2005 Ford," even if they share the same "beauty" level.
- **Loss Function Composition:**

$$\mathcal{L} = \underbrace{\text{Reconstruction Error}}_{\text{Keep car details}} + \underbrace{\beta \cdot \text{KL Divergence}}_{\text{Smooth design space}} + \underbrace{\gamma \cdot \text{GAN Loss}}_{\text{Photorealism}} + \underbrace{\delta \cdot \text{Prediction Error}}_{\text{Aesthetic Accuracy}}$$

- **Handling Subjectivity:** By modeling z as a **Gaussian distribution** (μ, σ), the model acknowledges that "beauty" has a range of uncertainty.

Data and Empirical Tests

Experimental Setup: The SUV Study

- **Image Data:** 180,000 images of vehicles from 2000-2018 (scraped from major automotive sites).
- **Consumer Data:** Ratings from a professional automotive "Theme Clinic." Consumers rated the "Overall Appeal" of 203 SUVs on a 1-10 scale.
- **Attributes (a):** Metadata including Brand (44 brands), Year (2000-2018), and Body Style.

The Cross-Validation

The authors also applied the model to **Dining Chairs** (using 11,000 images) to prove that the architecture isn't just a "car-model" but a general "aesthetic-model" applicable to any product category.

Key Results: Prediction & Generation

Result 1: Predictive Superiority

- **Performance Gap:** The proposed model improved predictive accuracy (R^2) by **43.5%** compared to standard Deep Learning models (like off-the-shelf CNNs).
- **Why it wins:** Standard CNNs are "data-hungry." They fail when you only have 200 labels. This model's semi-supervised approach uses the 180k unlabelled images to "pre-train" the visual brain.
- **Reliability:** The model's predictions highly correlate with the actual rankings produced by expensive consumer panels.

Table 2. Predictive Test of Machine Learning Augmentation vs. Baselines and Benchmarks

Prediction model	Mean absolute error (standard deviation)	Improvement
Baseline: Median rating in training images (constant rating)	0.620 (0.043)	0.0%
Benchmark: Computer vision features and random forest (conventional machine learning)	0.446 (0.047)	28.1%
Benchmark: VGG16 with fine-tuned final layers (pretrained deep learning)	0.405 (0.039)	34.7%
Proposed machine learning augmentation (custom deep learning)	0.350 (0.043)	43.5%

Result 2: Generative "Foresight"

One of the most impressive findings is the model's ability to "predict the future" of design:

- **The Experiment:** They trained the model *only* on data up to 2014.
- **The Task:** Ask the model to generate a "Futuristic SUV."
- **The Outcome:** The AI generated designs with features (like slim LED headlights and integrated grilles) that were **not common in 2014** but became the **industry standard in 2020**.
- **Implication:** Design evolution is not random. There are latent trajectories in aesthetics that AI can identify and project forward.

Practical Impact

Managerial Implications: Changing the Design Workflow

The paper suggests a three-step augmented workflow:

1. **Exploration:** The AI generates 1,000+ variations based on a brand's "DNA." Designers pick the top 50 that inspire them.
2. **Optimization:** Designers refine the 50. The AI provides a "Real-time Aesthetic Score." If a designer changes a curve, the score updates, showing if consumers might like it more or less.
3. **Defense:** When an engineer says "We must change this headlight to save $5," the designer can show the model's prediction: "This change will drop our aesthetic score by 12%, potentially costing us 2% in market share."

Summary Value

It turns "Design" from a *subjective argument* into a *quantifiable business asset*.

Conclusion and Takeaways

- **Key Contribution:** First major paper in Marketing Science to successfully integrate high-end Generative AI (GANs) with consumer preference modeling.
- **Efficiency:** Reduces the cost of design testing from $\$100,000$ to virtually $\$0$ for the initial screening phase.
- **Human-AI Collaboration:** The model is a "Co-pilot." It provides the quantitative "What" (what people like), while humans provide the qualitative "Why" and the final creative spark.

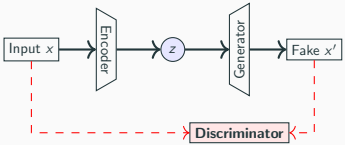
Replication Design: The "Classical" Approach

Methodology: The VAE-GAN Architecture

Objective: Replicate the *Burnap et al. (2023)* pipeline to quantify aesthetic value.

The Mechanism (Double Loop):

1. **Encoder (VAE):** Compresses an image x into a **Latent Embedding** z ("The DNA of the chair").
2. **Generator (GAN):** Decodes the embedding z back into an image to reconstruct the chair.
3. **Discriminator:** The "Judge" that compares the output to real photos and forces the Generator to create sharp, realistic images.



The Conflict

The **Encoder** wants z to be precise (to copy the input). The **Discriminator** forces the output to look like the training set. Balancing these opposing forces causes the instability we observed.

Implementation & Computational Reality

Hardware Setup:

- 4x NVIDIA RTX 4090 GPUs (Distributed Data Parallel).
- Training duration: $\sim$48 hours.

Technical Barriers Encountered:

1. **Gradient Explosion:** Variance ($Var(z)$) spiked to 5.9×10^6, causing model collapse.
2. **Instability:** Required custom "Drift Penalties" ($\epsilon = 10^{-5}$) to stabilize the Encoder.

Critical Technical Adjustment: The Drift Penalty

The Problem: The "Cheating" Encoder

- To minimize reconstruction error, the Encoder pushes latent values (z) apart towards infinity.
- *Consequence:* Variance explodes ($Var(z) \to \infty$), causing numerical overflow (NaNs) and model crashes.

The Fix: Drift Penalty (ϵ_{drift}) We added a regularization term to punish large z values:

$$\mathcal{L}_{drift} = \epsilon \cdot ||z||^2 \quad (\text{where } \epsilon = 10^{-5})$$

The Outcome: Stability vs. Generation Trade-off

- **Success:** It stopped the crash. Variance dropped from 10^7 to $\sim 3 \times 10^5$.
- **Limitation:** It was too weak to force z into a Standard Normal distribution ($\mathcal{N}(0, 1)$).
- *Result:* The model became stable enough to *reconstruct* ($z \approx 560$), but remained too spread out to *generate* from standard noise ($z \approx 1$).

Replication Results: The "White Screen" Phenomenon

Why do Reconstructions work but Sampling fails?

1. Reconstruction (Memorization)

Fig A: The model "photocopies" the input.

- **Result: Successful.**
- **Mechanism:** The Encoder outputs massive signals to minimize loss.
- **Evidence (Batch 3801):** Variance ($Var(z)$) exploded to **314,654**.

$$\text{Expected } z \approx \sqrt{314,654} \approx \pm\mathbf{560}$$

The Generator adapted to these "loud" signals.

2. Random Sampling (Generation)

[Pure White Output]

Fig B: The model cannot invent new designs.

- **Result: Failed (Mode Collapse).**
- **Root Cause:** We feed standard noise ($z \sim \mathcal{N}(0, 1)$).
- **The Mismatch:**

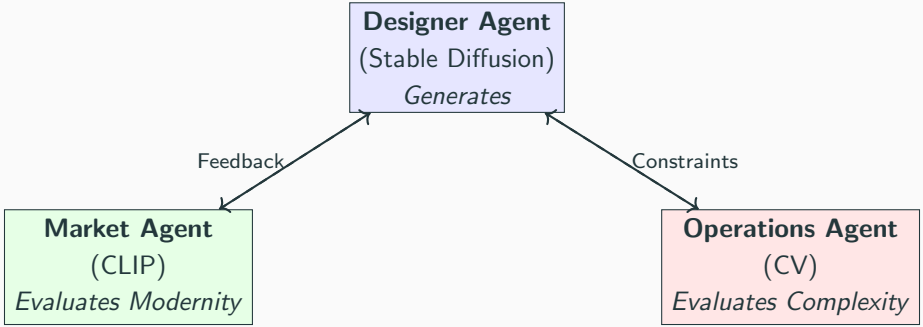
$$\frac{\text{Input } (1)}{\text{Expected } (560)} \approx \mathbf{0.17\%}$$

The signal is too weak to activate the features, resulting in a default bias output (White Screen).

Extension: An Agentic AI Workflow

The Pivot: From Training to Agents

Proposal: Replace the unstable VAE-GAN with a **Foundation Model Agent System**.



The Market Agent: CLIP (Contrastive Language-Image Pre-training)

What is CLIP?

- A Foundation Model trained by OpenAI on 400M image-text pairs.
- It aligns images and text in a shared embedding space.

Our Algorithm (Zero-Shot Ensemble): Instead of training a regressor, we define a **5-Point Semantic Scale**:

- **Anchors:** We map 5 text prompts (e.g., *"Traditional Antique"* $\rightarrow$ 1.0, *"Futuristic Modern"* $\rightarrow$ 5.0) to the Likert scale.
- **Scoring:** We compute the **Expected Value** of the rating:

$$\text{Score} = \sum_{i=1}^{5} (P(\text{prompt}_i | \text{Image}) \times \text{ScaleValue}_i)$$

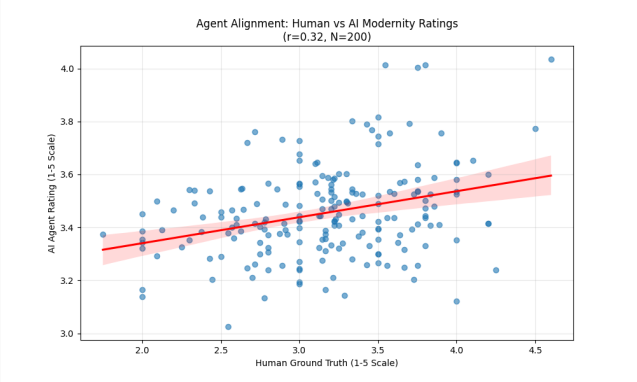
- **Robustness:** To handle 2D occlusions, we average probabilities across **4 camera views** $(0^\circ, 15^\circ, 30^\circ, 45^\circ)$.

Validation 1: Market Agent Accuracy

Result: Significant positive correlation with human perception.

- $r = 0.3167$ (Pearson)
- $p < 0.001$ (Significant)

Implication: The AI agent can validly screen 10,000+ designs for aesthetic appeal without human intervention.



The Operations Agent: Measuring Manufacturing Complexity

To automate cost estimation, we use Computer Vision to proxy **Manufacturing Complexity**.

Metric: Structural Edge Density (Canny)

- **Method:** We apply the Canny algorithm to detect all structural edges (contours) in the chair image.
- **The Economic Logic:**

  More Edges $\approx$ More Cuts, Joints, & Labor

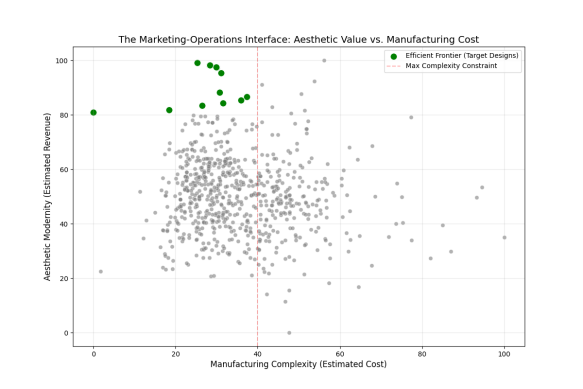
  A simple plywood chair has few edges. A woven or ornate chair has thousands.
- **Scoring:** We normalize the pixel-edge-sum to a **0-100 Difficulty Index**.

$\downarrow$ *Canny Filter* $\downarrow$

[Edge Map]

(Low Complexity $\rightarrow$ Low Cost)

Validation 2: The Efficient Frontier



Defining the "Sweet Spot"

- We filter for designs that meet **dual strategic constraints**:
  1. **Marketing:** Modernity > 80.
  2. **Operations:** Complexity < 40.

Strategic Outcome:

- **Green Zone:** Pareto-Efficient Designs.
- **Filtration:** The agent automatically reduced 700 candidates to the most profitable options.

Visual Results: Automated Design Discovery

Efficient Design
(Existing in Catalog)

Human rated Modernity: **NA**
AI rated Modernity: **3.8/5.0**
Complexity: **18.4/100.00**

Complex Design
(Existing in Catalog)

Human rated Modernity: **3.4/5.0**
AI rated Modernity: **3.3/5.0**
Complexity: **100.0/100.0**

Agent Generated
(Stable Diffusion)

Synthetic Prototype
Prompt: "a ultra-modern futuristic chair, gray scale, 8k".

Conclusion

Summary

1. **Replication:** We validated the technical barriers of pixel-level training (instability, compute cost) for OM research.
2. **Extension:** We successfully demonstrated a **Foundation Model Agent** approach.
3. **Contribution:**
   - **Market Agent:** Validated aesthetic scoring ($r = 0.31$).
   - **Ops Agent:** Quantified the design-cost trade-off.
   - **Workflow:** A closed-loop system for automated product discovery.

Thank you!

Questions?

Appendix A: Technical Fixes for Replication Stability

1. Drift Penalty (Preventing Variance Explosion)

```
# In train.py (Encoder Step)
# We penalize the encoder if raw outputs (real_h) get too large.
# Tuned from 0.001 -> 0.00001 based on log_131825 failure analysis.

drift_loss = (real_h ** 2).mean() * 0.00001
e_losses.append(drift_loss)
```

2. Numerical Stability Fixes (Losses & Gradients)

```
# A. Fix CUDA Assertion Error in Mask Loss
# Squash Tanh output (-1, 1) to Sigmoid (0, 1) range
mask_loss = mask_loss_criterion(torch.sigmoid(recon_mask), batch_masks)

# B. Fix Gradient Clipping for DDP
# Direct access to model parameters bypasses generator graph detachment
if labeled_data:
    p_loss.backward()
    torch.nn.utils.clip_grad_norm_(encoder.parameters(), max_norm=1.0)
    torch.nn.utils.clip_grad_norm_(predictor.parameters(), max_norm=1.0)
```

Appendix A (continued): Basic Code Adjustments

3. Latent Space Sampling (Spherical Normalization) Standard ProGANs often perform better when sampling from a hypersphere. We enforced normalization on the random noise z:

```
# In config.py
c.noise = 'sphere'  # Enforce spherical latent space configuration

# In evaluate.py (generate_intermediate_samples)
# Generate standard Gaussian noise
myz = Variable(torch.randn(colN, session.c.nz)).cuda()

# PROJECT ONTO SPHERE: Normalize z before feeding to Generator
# This matches the training distribution expected by ProGAN
myz = training_utils.normalize(myz)
```

4. Stability & Compatibility Fixes

```
# In config.py: Prevent GPU OOM (Out of Memory) fragmentation
c.empty_cache = True

# In evaluate.py: Fix for newer torchvision versions
torchvision.utils.save_image(
    torch.cat(images, 0),
    save_path,
    nrow=colN,
    normalize=True,
    value_range=(-1, 1), # Replaced deprecated 'range=(-1,1)'
    padding=0
)
```

Appendix B: Market Agent Logic (CLIP Scoring)

Algorithm: Weighted Semantic Scoring

```
# 1. Define Semantic Anchors for 5-Point Scale
SCALE_MAPPING = {
    "a-photo-of-a-very-traditional-antique-chair": 1.0,
    "a-photo-of-a-traditional-chair": 2.0,
    "a-photo-of-a-standard-chair": 3.0,
    "a-photo-of-a-modern-chair": 4.0,
    "a-photo-of-a-futuristic-modern-chair": 5.0
}

# 2. Multi-View Ensemble (Robustness)
# We loop through 4 angles (0, 15, 30, 45) to get a stable score
view_probs = []
for v in VIEWS:
    logits = model(text=PROMPTS, images=img_view[v])
    probs = logits.softmax(dim=1) # Convert to probability
    view_probs.append(probs)

# 3. Calculate Expected Value (Weighted Average)
avg_probs = np.mean(view_probs, axis=0)
ai_rating = np.sum(avg_probs * [1.0, 2.0, 3.0, 4.0, 5.0])
```

Appendix C: Operations Agent Logic (Complexity)

Algorithm: Manufacturing Proxy (Edge Density)

```
def calculate_complexity(img_array):
    # 1. Edge Density (Primary Metric)
    # Canny detects structural lines (cuts/joints)
    edges = cv2.Canny(img_array, 100, 200)
    edge_score = np.sum(edges) / 255.0

    return edge_score

# Analysis Loop
complexities = []
for img in dataset:
    # We quantify complexity based on edge density
    edge_score = calculate_complexity(img)
    complexities.append(edge_score)

# Final Normalization (0-100 Scale)
df['ops_score'] = (raw - min) / (max - min) * 100
```